

```
process/.style
process/.style//
processgreen/.style
processgreen/.style//
processorange/.style
processorange/.style//
processpurple/.style
processpurple/.style//
entity/.style
entity/.style//
datastore/.style
datastore/.style//
arrow/.style
arrow/.style//
dashedarrow/.style
dashedarrow/.style//
relation/.style
relation/.style//
```

UCS503 – Software Engineering Project

Academic Year 2026–2027

CodeRoom

Real-Time Collaborative Code Editor

A Web-Based Platform for Collaborative Coding and Real-Time Code Synchronization

Submitted by:

Yuvraj Jasuja Team Lead, Full-Stack Architect

Harwinder Singh Frontend Developer and UI/UX Specialist

Bachelor of Engineering in Computer Engineering

Submitted to:

Department of Computer Science and Engineering

Mid-Semester Evaluation Report

Thapar Institute of Engineering and Technology, Patiala

2026–2027

Contents

1	Introduction	1
1.1	Background and Context	1
1.2	Problem Statement	1
1.3	Project Objectives	1
1.4	Project Scope	2
2	Methodology and System Design	3
2.1	Development Methodology	3
2.2	System Architecture	3
2.2.1	High-Level Architecture	3
2.2.2	Technical Stack	4
2.3	System Workflow	4
3	Data Flow Diagrams	5
3.1	Introduction to DFD	5
3.2	DFD Level 0 – Context Diagram	5
3.3	DFD Level 1 – Main Processes	5
3.4	DFD Level 2 – Code Synchronization	5
4	Entity Relationship Diagram	6
4.1	Introduction	6
4.2	Entities and Attributes	6
4.2.1	User	6
4.2.2	Room	6
4.2.3	Code Session	6
4.3	ER Diagram	7
4.4	Relationship Description	7
5	Implementation and Module Design	8
5.1	Module 1: Room Management	8
5.2	Module 2: Real-Time Code Synchronization	8
5.3	Module 3: Frontend Interface and Client List	9
5.4	Module 4: Documentation and Repository Management	9
6	Testing and Evaluation	10
6.1	Testing Methodology	10
6.1.1	Unit Testing	10
6.1.2	Integration Testing	10
6.1.3	System Testing	10
6.2	Testing Results	11
6.3	Testing Framework	11
6.4	Limitations	11
7	Weekly Progress Journal	12
7.1	Weekly Progress Journal – Yuvraj Jasuja	12
7.2	Weekly Progress Journal – Harwinder Singh	13

8	Conclusion and Future Work	15
8.1	Summary of Work	15
8.2	Key Findings	15
8.3	Future Work	15
8.4	Final Remarks	16

Chapter 1

Introduction

1.1 Background and Context

In modern software development, programmers often work together on the same codebase from different locations. Traditional code editors generally provide an individual coding environment, while collaboration requires external communication tools, file sharing, or version control systems.

These methods can make real-time collaboration difficult. Team members may need to share code manually, wait for updates, or face difficulties while debugging together.

CodeRoom is a web-based real-time collaborative code editor designed to allow multiple users to work on the same code file simultaneously. It provides a shared coding environment where users can create or join coding rooms and synchronize code changes through real-time communication.

The project uses React for the frontend, Node.js and Express.js for the backend, Socket.IO for real-time communication, and CodeMirror for code editing.

1.2 Problem Statement

Traditional code editors do not provide a simple built-in environment for multiple users to edit the same code simultaneously.

The main problems include:

1. Difficulty in sharing code changes instantly.
2. Lack of a common real-time coding environment.
3. Delays during collaborative debugging.
4. Manual sharing of files and code snippets.
5. Difficulty in tracking connected collaborators.

CodeRoom addresses these problems by providing a shared coding room where users can join using a room ID and collaborate through synchronized code editing.

1.3 Project Objectives

The main objectives of CodeRoom are:

1. Develop a real-time collaborative code editor.
2. Allow users to create and join coding rooms.
3. Synchronize code changes between connected users.
4. Display the list of active collaborators.
5. Provide a simple and responsive frontend interface.
6. Implement real-time communication using Socket.IO.
7. Maintain a modular and maintainable full-stack architecture.

1.4 Project Scope

The current prototype focuses on:

- Room creation and joining.
- Real-time code synchronization.
- CodeMirror editor integration.
- Connected user display.
- Room ID sharing.
- Leaving active coding sessions.
- Basic frontend and backend integration.

Advanced features such as authentication, code execution, persistent code storage, and integrated voice communication may be considered for future development.

Chapter 2

Methodology and System Design

2.1 Development Methodology

The project follows an iterative development approach. The system was developed in stages, beginning with project ideation and architecture, followed by environment setup, room management, real-time synchronization, frontend refinement, and testing.

The major development stages were:

1. Project inception and architecture planning.
2. GitHub repository and development environment setup.
3. Protocol design and project presentation.
4. Room management and frontend integration.
5. Real-time Socket.IO and CodeMirror integration.
6. UI refinement, documentation, and testing.

2.2 System Architecture

2.2.1 High-Level Architecture

CodeRoom follows a modular client-server architecture. The frontend provides the user interface and code editor, while the backend manages room connections and real-time communication.

The system consists of three main layers:

1. **Client Interface Layer:** React, Vite, CodeMirror, and frontend components.
2. **Application Logic Layer:** Node.js, Express.js, and Socket.IO.
3. **Data and Session Layer:** Room state, connected users, and synchronized code data.

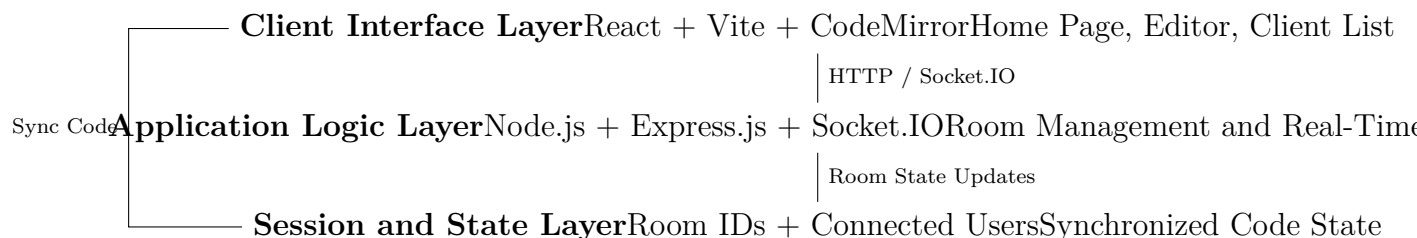


Figure 2.1: CodeRoom High-Level System Architecture

2.2.2 Technical Stack

The following technologies are used in the CodeRoom prototype:

Technology	Purpose
React	Frontend user interface
Vite	Frontend development and build tool
Node.js	Backend runtime
Express.js	HTTP server and API foundation
Socket.IO	Real-time communication
CodeMirror	Code editor component
JavaScript	Application development
CSS	Styling and responsive layout
Git and GitHub	Version control and collaboration
MkDocs	Project documentation
GitHub Actions	Documentation build automation

Table 2.1: CodeRoom Technology Stack

2.3 System Workflow

The basic workflow of CodeRoom is:

1. The user opens the CodeRoom website.
2. The user creates a new room or joins an existing room.
3. The frontend connects to the backend using Socket.IO.
4. The user enters the shared code editor.
5. Code changes are sent to other connected users.
6. Other users receive the updated code.
7. The active client list is updated when users join or leave.

Chapter 3

Data Flow Diagrams

3.1 Introduction to DFD

A Data Flow Diagram (DFD) represents how data moves through a system. It shows external entities, processes, data stores, and the flow of information.

CodeRoom uses DFDs to describe the interaction between users, the collaborative editor, room management, and real-time synchronization.

3.2 DFD Level 0 – Context Diagram

At Level 0, CodeRoom is represented as a single system process. The diagram shows the external users and the information exchanged with the system.



Figure 3.1: CodeRoom DFD Level 0 – Context Diagram

3.3 DFD Level 1 – Main Processes

Level 1 divides CodeRoom into its main processes. It shows how users interact with the room management process and the real-time synchronization process.

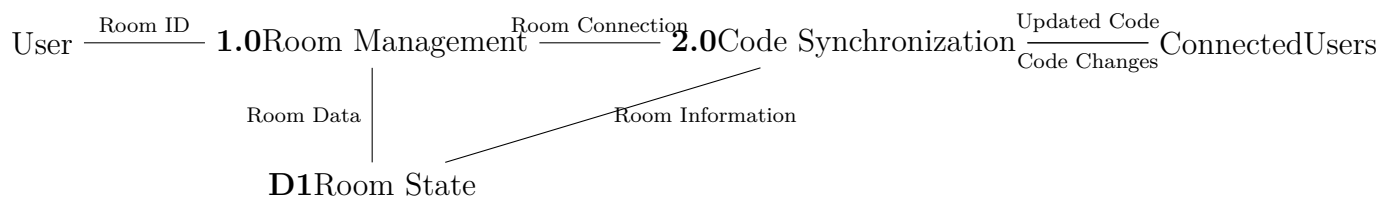


Figure 3.2: CodeRoom DFD Level 1

3.4 DFD Level 2 – Code Synchronization

The code synchronization process can be further divided into smaller processes.



Figure 3.3: CodeRoom DFD Level 2 – Code Synchronization

Chapter 4

Entity Relationship Diagram

4.1 Introduction

An Entity Relationship Diagram (ERD) represents the entities of a system and the relationships between them.

For the CodeRoom prototype, the main entities are User, Room, and Code Session. The diagram models the logical data required for collaborative coding.

4.2 Entities and Attributes

4.2.1 User

The User entity represents a participant in the collaborative coding environment.

Attributes:

- User ID – Primary Key
- Display Name
- Socket ID

4.2.2 Room

The Room entity represents a shared coding workspace.

Attributes:

- Room ID – Primary Key
- Room Name
- Created At

4.2.3 Code Session

The Code Session entity represents the shared coding state of a room.

Attributes:

- Session ID – Primary Key
- Room ID – Foreign Key
- Source Code
- Language
- Last Updated

4.3 ER Diagram

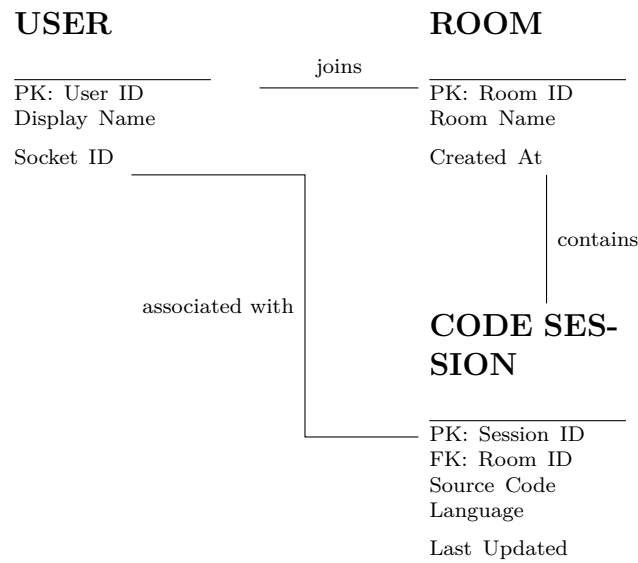


Figure 4.1: CodeRoom Entity Relationship Diagram

4.4 Relationship Description

Relationship	Type	Description
User – Room	Many-to-Many (logical)	Multiple users can join a room, and a user may join multiple rooms.
Room – Code Session	One-to-One (logical)	A room maintains its shared coding session in the prototype model.
User – Code Session	Participates	Users participate in coding sessions through their room connection.

Table 4.1: CodeRoom Entity Relationships

Note: The current prototype primarily manages room and client state through Socket.IO. The ER diagram represents the logical data model for the collaborative editor and can be implemented using persistent storage in a future version.

Chapter 5

Implementation and Module Design

5.1 Module 1: Room Management

The Room Management module allows users to create or join a coding room.

The module performs the following operations:

1. Generate a unique room ID.
2. Accept an invitation code from a user.
3. Navigate users to the editor route.
4. Connect the user to the room using Socket.IO.
5. Display notifications for invalid room input.

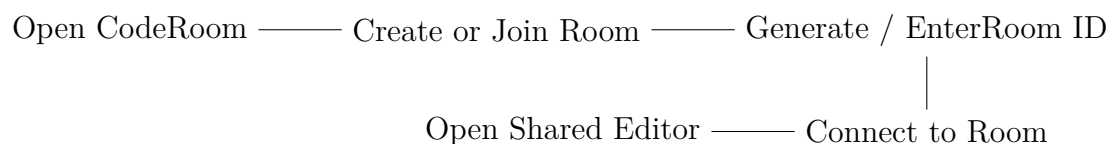


Figure 5.1: Room Management Flow

5.2 Module 2: Real-Time Code Synchronization

The Real-Time Code Synchronization module is the core component of CodeRoom. It allows users to share code changes with other connected clients.

The module uses Socket.IO events:

- JOIN – Request to join a room.
- JOINED – Confirmation of successful room joining.
- CODE_CHANGE – Sends updated code.
- SYNC_CODE – Receives synchronized code.
- DISCONNECTED – Handles user disconnection.



Figure 5.2: Real-Time Code Synchronization Flow

5.3 Module 3: Frontend Interface and Client List

The frontend interface is developed using React and CodeMirror.

The main frontend components include:

- **Home.js:** Room creation, joining, and input validation.
- **Editor.js:** CodeMirror editor integration and code changes.
- **Client.js:** Display of connected user avatars.
- **App.js:** Main application structure and routing.
- **App.css and index.css:** Application styling and layout.

The client list is updated whenever a user joins or leaves the room. This provides basic visibility of active collaborators.

5.4 Module 4: Documentation and Repository Management

The project uses GitHub for version control and collaboration.

Documentation and repository activities include:

- Maintaining the project repository.
- Organizing frontend and backend modules.
- Maintaining the README file.
- Preparing project documentation using MkDocs.
- Configuring GitHub Actions for documentation builds.
- Maintaining the project directory structure.

Chapter 6

Testing and Evaluation

6.1 Testing Methodology

Testing was performed to verify the correctness of the CodeRoom prototype. The testing process focused on frontend functionality, backend communication, room management, and real-time code synchronization.

6.1.1 Unit Testing

Unit testing focuses on individual components and functions.

The following components were considered:

- Room ID generation.
- Room joining validation.
- Frontend input handling.
- Code editor event handling.
- Client list rendering.

6.1.2 Integration Testing

Integration testing verifies communication between the frontend and backend.

The following interactions were tested:

- React frontend with Express backend.
- Socket.IO client and server communication.
- Room joining and connection events.
- Code changes between multiple clients.
- Client connection and disconnection updates.

6.1.3 System Testing

System testing verifies the complete application workflow.

The following scenarios were considered:

- Creating a coding room.
- Joining a room using a room ID.
- Editing code in the shared editor.
- Synchronizing code with another browser client.
- Displaying connected collaborators.
- Leaving a coding session.

6.2 Testing Results

Test Case	Expected Result	Re-	Status
Create Room	Unique room is generated	is	Passed
Join Room	User enters selected room	se-	Passed
Room ID Validation	Invalid input is handled	is	Passed
Socket Connection	Client connects to server		Passed
Code Synchronization	Code updates are shared		Passed
Client List Update	Active users are displayed	are	Passed
User Disconnection	Client list is updated	up-	Passed
Room ID Copy	Room ID can be copied	be	Passed
Responsive UI	Layout adapts to screen size		Passed

Table 6.1: CodeRoom Testing Results

6.3 Testing Framework

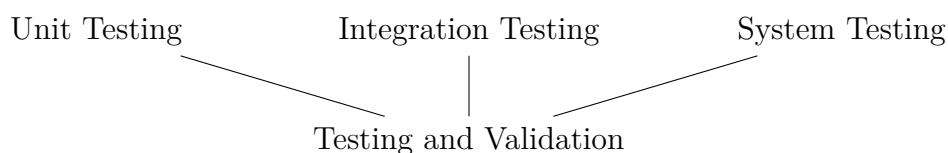


Figure 6.1: CodeRoom Testing and Evaluation Framework

6.4 Limitations

The current prototype has the following limitations:

1. Code sessions are primarily managed through real-time room state.
2. Persistent code storage is not included in the current prototype.
3. Authentication and role-based access are not fully implemented.
4. The editor currently focuses on JavaScript coding.
5. Conflict resolution for simultaneous edits is limited.
6. Automated code execution is outside the current scope.

Chapter 7

Weekly Progress Journal

7.1 Weekly Progress Journal – Yuvraj Jasuja

Week 1: August 3 – August 9

Project Inception and Architecture

- Formed the team for the UCS503 Software Engineering project.
- Brainstormed and finalized CodeRoom as the project topic.
- Researched the MERN stack and real-time communication technologies.
- Planned the synchronization logic and UI architecture.

Week 2: August 10 – August 16

Git Setup and Project Environment

- Initialized the GitHub repository.
- Established frontend and backend development environments.
- Configured the Express.js server foundation.
- Set up HTTP and Socket.IO server communication.
- Verified backend connectivity.

Week 3: August 17 – August 23

Protocol Design and Pitch Presentation

- Defined real-time events such as JOIN, JOINED, CODE_CHANGE, and SYNC_CODE.
- Contributed to the project pitch presentation.
- Planned the real-time code synchronization MVP.
- Configured the MkDocs documentation environment.
- Set up the GitHub Actions documentation pipeline.

Week 4: August 24 – August 30

Room Management and Frontend Integration

- Built the initial frontend architecture.
- Implemented room creation using UUID.
- Implemented invitation code joining.
- Added toast notifications.
- Developed room routing using `/editor/:roomId`.
- Managed repository operations and `.gitignore` configuration.

Week 5: August 31 – September 6

Real-Time Socket Engine and CodeMirror

- Developed the real-time code synchronization engine.
- Integrated Socket.IO events for code changes.
- Integrated CodeMirror with JavaScript language mode.
- Implemented client connection and disconnection broadcasts.
- Added room ID copying functionality.
- Resolved repository merge conflicts.

Week 6: September 7 – September 13

Final Polish and Repository Standardization

- Restructured the project directory.
- Refined frontend and editor UI components.
- Removed outdated branding and logo references.
- Performed multi-client browser verification.
- Created the project Makefile and pyproject.toml.
- Updated README and MkDocs documentation.

7.2 Weekly Progress Journal – Harwinder Singh

Week 1: August 3 – August 9

Project Inception and UI Wireframing

- Participated in project ideation and requirements gathering.
- Researched frontend libraries for the collaborative editor.
- Studied CodeMirror, react-hot-toast, react-avatar, and react-router-dom.
- Planned the frontend user experience.

Week 2: August 10 – August 16

React Infrastructure and Styling

- Set up the React application under the frontend directory.
- Configured required frontend dependencies.
- Designed global CSS theme tokens.
- Implemented font hierarchy and responsive containers.
- Updated the HTML baseline template.

Week 3: August 17 – August 23

Component Architecture and Layout

- Created the modular frontend component hierarchy.
- Implemented the Client.js component.
- Designed the sidebar for connected users.
- Added status indicators and action controls.
- Contributed to project presentation and documentation.

Week 4: August 24 – August 30

Form Validation and UI Interactions

- Implemented form controls on Home.js.
- Added input validation notifications.
- Implemented Enter key navigation for room joining.
- Tested responsive behavior across screen sizes.

Week 5: August 31 – September 6

CodeMirror and Client List Integration

- Developed the Editor.js component.
- Integrated CodeMirror initialization and event handling.
- Connected real-time client list updates.
- Added room ID copying and leave-session controls.

Week 6: September 7 – September 13

UI Polish and Quality Assurance

- Removed legacy logo assets.
- Updated the home and editor page styling.
- Performed multi-client browser testing.
- Verified avatar updates and code synchronization.
- Organized project assets and documentation.

Chapter 8

Conclusion and Future Work

8.1 Summary of Work

CodeRoom is a real-time collaborative code editor developed as a Software Engineering project. The system provides a shared environment where users can create or join coding rooms and collaborate through synchronized code editing.

During the prototype development phase, the team completed the main system design, frontend structure, room management, real-time Socket.IO communication, and CodeMirror integration.

The project also included documentation, repository organization, and multi-client browser testing.

8.2 Key Findings

1. **Real-Time Collaboration:** Socket.IO enables communication between connected users and supports code synchronization.
2. **Modular Frontend:** React components provide a structured and maintainable frontend.
3. **Room-Based Architecture:** Room IDs allow users to enter a shared coding environment.
4. **Improved Code Sharing:** Users can collaborate without manually exchanging code files.
5. **Documentation and Version Control:** GitHub and MkDocs support project collaboration and documentation.

8.3 Future Work

The following features may be implemented in the final phase:

1. User authentication and authorization.
2. Persistent storage of coding sessions.
3. Support for multiple programming languages.
4. Integrated code execution.
5. Improved simultaneous editing conflict handling.
6. Cursor and selection sharing.
7. Chat functionality inside coding rooms.
8. Deployment on a cloud platform.
9. Improved security and room access control.

8.4 Final Remarks

CodeRoom demonstrates the application of full-stack development and real-time communication in collaborative software engineering.

The current prototype establishes the basic foundation for a shared coding environment. Future development can extend the system with persistent storage, authentication, advanced editing capabilities, and integrated development tools.

Bibliography

- [1] Meta Platforms, Inc., *React Documentation*. Available at: <https://react.dev/learn>
- [2] OpenJS Foundation, *Node.js Documentation*. Available at: <https://nodejs.org/docs/latest/api/>
- [3] Express.js, *Express.js Documentation*. Available at: <https://expressjs.com/>
- [4] Socket.IO, *Socket.IO Documentation*. Available at: <https://socket.io/docs/v4/>
- [5] CodeMirror, *CodeMirror Documentation*. Available at: <https://codemirror.net/>
- [6] Vite Team, *Vite Documentation*. Available at: <https://vite.dev/guide/>
- [7] GitHub, *GitHub Documentation*. Available at: <https://docs.github.com/>